// T-OLIMPIANS — PREZENTACJA DO LEKCJI

Dziel i zwyciężaj

Zamiast zmniejszać problem o jeden — dzielimy go na połowy. Szybkie potęgowanie, wyszukiwanie binarne, scalanie i sortowanie przez scalanie, zliczanie inwersji, quickselect, najlepszy fragment i fraktale.

C++ · REKURENCJA · CZĘŚĆ 6 Z 6 · PO CZĘŚCIACH 1-5



PLAN PREZENTACJI

01 Idea: dziel i zwyciężaj
02 Rozgrzewka: min i max
03 Szybkie potęgowanie
04 Wyszukiwanie binarne
05 Scalanie dwóch ciągów
06 Sortowanie przez scalanie
07 Ile to kosztuje?
08 Zliczanie inwersji
09 Quicksort i quickselect
10 Najlepszy spójny fragment
11 Fraktale i napisy
12 Sprawdź się
13 Podsumowanie

01 Dziel i zwyciężaj

Podziel problem na dwie połowy, rozwiąż każdą rekurencyjnie, a potem połącz wyniki.

1 · podziel

Tablicę, przedział albo liczbę dzielimy na dwie (mniej więcej) równe części.

2 · rozwiąż

Każdą część rozwiązuje wywołanie rekurencyjne. Bardzo mała część to warunek stopu.

3 · połącz

Z wyników obu części budujemy wynik dla całości. Często to najciekawszy krok.

Przykład z życia: posortować stos 1000 klasówek? Podziel na dwa stosy po 500, daj je dwóm osobom do posortowania, a potem **scal** dwa posortowane stosy w jeden.

01 Po co dzielić na pół, a nie zmniejszać o 1?

Do tej pory problem malał **o jeden**: $n \to n-1 \to n-2 \to \dots$ Rekurencja miała **n poziomów**.

Gdy dzielimy na pół, problem maleje **dwa razy** na każdym poziomie: $n \to n/2 \to n/4 \to \dots \to 1$.

> Liczba poziomów to tyle, ile razy da się podzielić n przez 2 — czyli **$\log_2 n$**. Nawet dla miliarda to tylko **30** poziomów.

n	zmniejszanie o 1	dzielenie na pół
8	8 poziomów	3 poziomy
1 000	1 000	10
1 000 000	milion	20
10^9	miliard	30
10^{18}	—	60

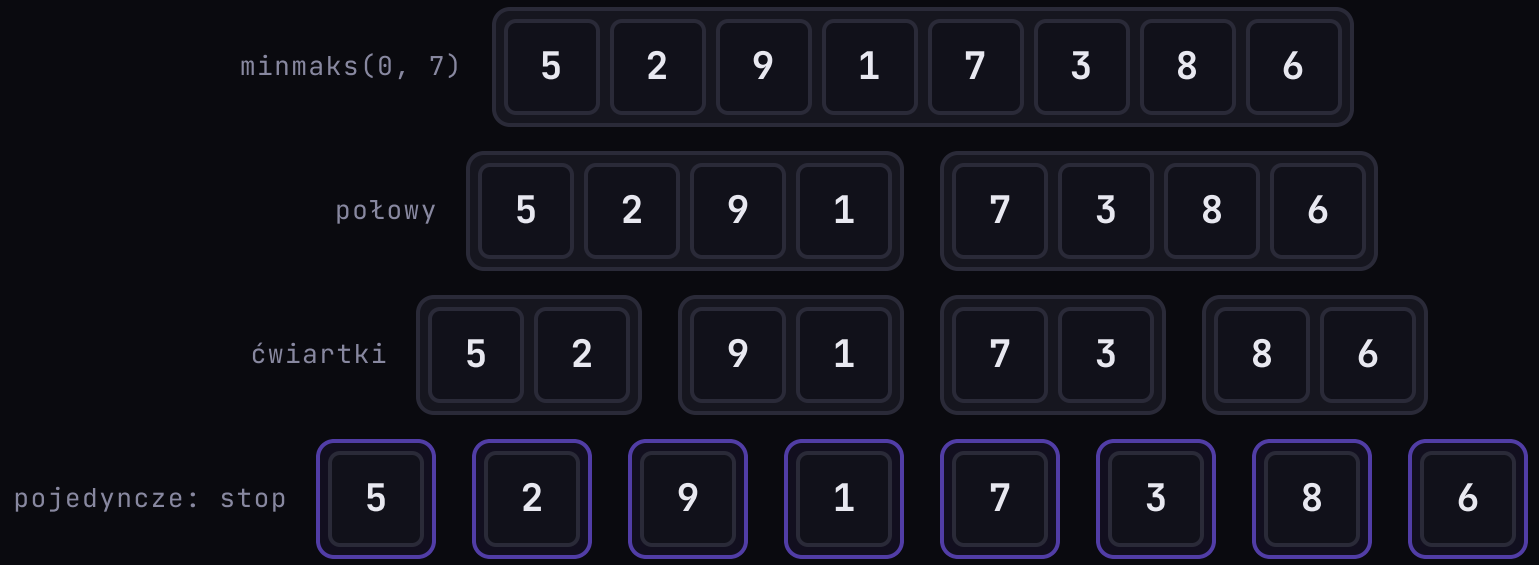
02 Minimum i maksimum — dzielimy tablicę

```
pair<int, int> minmaks(int l, int r) {
    if (l == r) return {a[l], a[l]};
    int s = (l + r) / 2;
    auto L = minmaks(l, s);
    auto R = minmaks(s + 1, r);
    return {min(L.first, R.first),
            max(L.second, R.second)};
}
```

1. **stop**: jeden element jest swoim minimum i maksimum,
2. **podziel**: środek `s = (l + r) / 2` — lewa połowa l…s, prawa s + 1…r,
3. **połącz**: mniejsze z dwóch minimów, większe z dwóch maksimów.

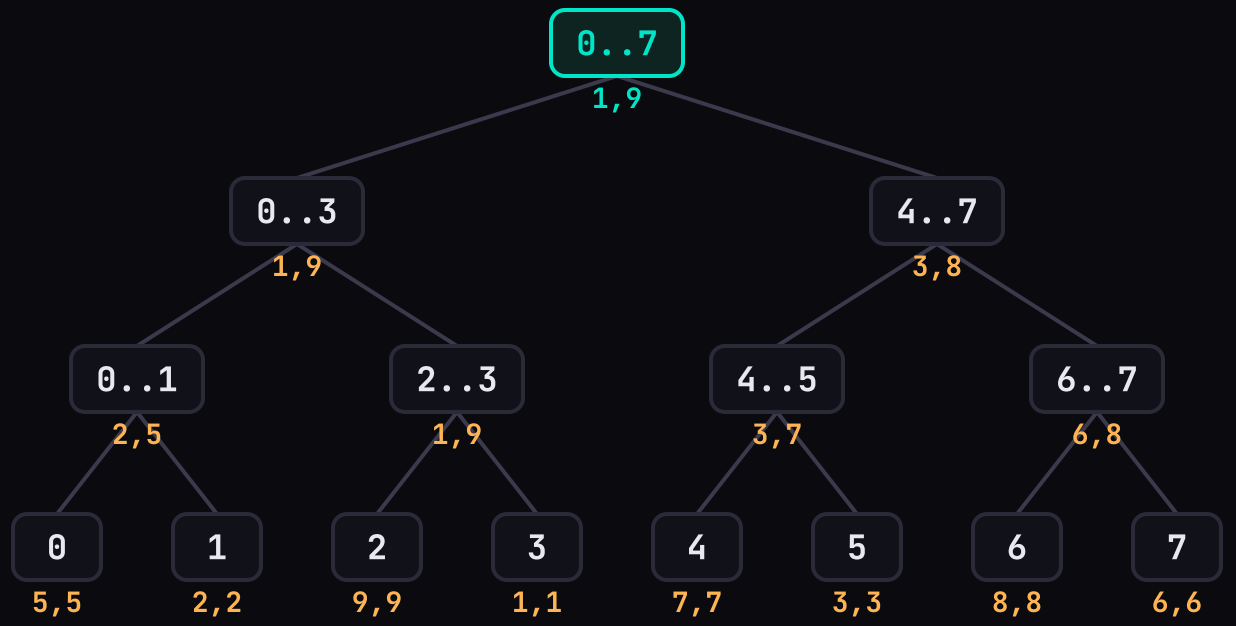
Funkcja zwraca **parę** (pair) — dwie liczby naraz. `L.first` to minimum lewej połowy, `L.second` — maksimum.

02 Podział tablicy na kawałki



Każdy kawałek dzieli się na dwa, aż zostaną pojedyncze elementy. Potem wyniki **łączą się w górę**: (5,5) i (2,2) → (2,5); (2,5) i (1,9) → (1,9); … aż do całej tablicy: **(1, 9)**.

02 Drzewo wywołań `minmaks`



W węzłach — przedziały indeksów, pod nimi — zwrócona para (min, max). Drzewo ma tylko **$\log_2 8 = 3$** poziomy podziałów.

03 Potęgowanie: zwykłe jest za wolne

Z CZĘŚCI 2 — N WYWOŁAŃ

```
long long potega(long long a, long long n) {
    if (n == 0) return 1;
    return a * potega(a, n - 1);
}
```

Uwaga: Dla $n = 10^{18}$ to 10^{18} wywołań — nie skończy się do końca świata (i dużo wcześniej zabraknie stosu).

Pomysł: **$a^{100} = (a^{50})^2$**. Wystarczy policzyć a^{50} **raz** i podnieść do kwadratu. A $a^{50} = (a^{25})^2$, a $a^{25} = a \cdot (a^{12})^2$…

n	wzór
parzyste	$a^n = (a^{n/2})^2$
nieparzyste	$a^n = a \cdot (a^{(n-1)/2})^2$
0	$a^0 = 1$

03 Przykład: 3^{13}

wywołanie	policz	wynik
`potega(3, 13)`	$3 \cdot (3^6)^2$	$3 \cdot 729^2 =$ **1 594 323**
`potega(3, 6)`	$(3^3)^2$	$27^2 = 729$
`potega(3, 3)`	$3 \cdot (3^1)^2$	$3 \cdot 9 = 27$
`potega(3, 1)`	$3 \cdot (3^0)^2$	$3 \cdot 1 = 3$
`potega(3, 0)`	stop	1

Wykładnik maleje **o połowę**: $13 \to 6 \to 3 \to 1 \to 0$.
Zamiast 13 mnożeń — **5 wywołań**.

> Dla $n = 10^{18}$ wystarczy ok. **60 wywołań**.

03 Szybkie potęgowanie modulo m

```
long long m;

long long potega(long long a, long long n) {
    if (n == 0) return 1 % m;
    long long p = potega(a, n / 2);
    p = p * p % m;
    if (n % 2 == 1) p = p * (a % m) % m;
    return p;
}
```

Wyniki potęg są gigantyczne, więc w zadaniach liczy się je **modulo m** — resztę z dzielenia przez m. Bierzemy resztę po każdym mnożeniu, żeby liczby nie urosły.

Uwaga: Linijka 5: `potega(a, n / 2)` wywołujemy **jeden raz** i zapisujemy w zmiennej. Napisanie `potega(a, n/2) * potega(a, n/2)` zrobiłoby z powrotem n wywołań!

Gdy $m \le 10^9$, iloczyn dwóch reszt jest mniejszy niż 10^{18} — mieści się w long long.

04 Wyszukiwanie binarne — zgadnij liczbę

Gra: myślę o liczbie od 1 do 100, a Ty zgadujesz. Na każdą próbę mówię „więcej” albo „mniej”. Najlepiej pytać o **środek** — każda odpowiedź wyrzuca **połowę** możliwości.

Wystarczy **7 pytań**: 100 → 50 → 25 → 13 → 7 → 4 → 2 → 1.

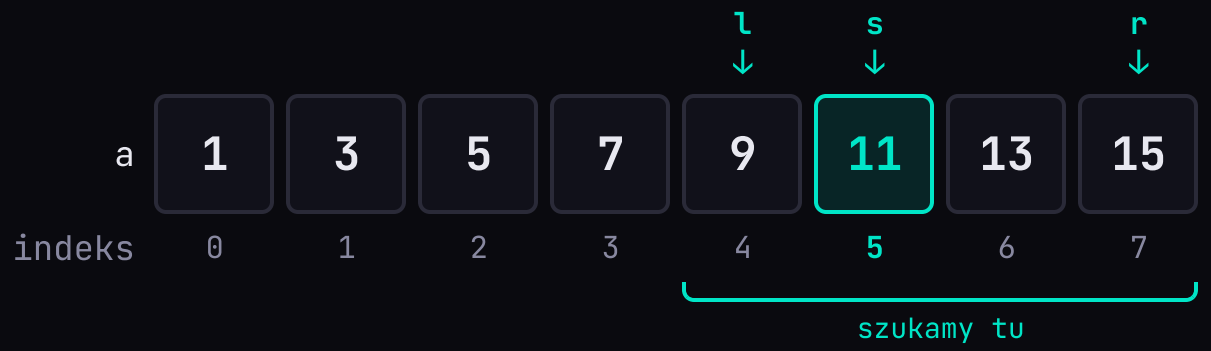
Tak samo szukamy liczby x w **posortowanej** tablicy: patrzymy na środkowy element.

1. środek == x — **znalezione**,
2. środek > x — x może być tylko w **lewej** połowie,
3. środek < x — tylko w **prawej**.

04 Szukamy x = 11



l = 0, r = 7, środek s = 3: a[3] = 7 < 11 → szukamy w prawej połowie (4…7).



l = 4, r = 7, s = 5: a[5] = 11 == 11 → **znalezione** na indeksie 5.

04 Wyszukiwanie binarne — kod

```
int szukaj(int l, int r, int x) {
    if (l > r) return -1;          // pusto: nie ma
    int s = (l + r) / 2;
    if (a[s] == x) return s;
    if (a[s] > x) return szukaj(l, s - 1, x);
    return szukaj(s + 1, r, x);
}
```

Tylko **jedno** wywołanie rekurencyjne — do tej połowy, w której x może być. Druga połowa jest odrzucana w całości.

n	najwyżej kroków
1 000	10
1 000 000	20
10^9	30

Uwaga: Działa **tylko na posortowanej tablicy**! W kompendium o wyszukiwaniu binarnym są osobne tematy — to dopiero wstęp.

05 Scalanie dwóch posortowanych ciągów



Najmniejszy element wyniku to **mniejszy z pierwszych** elementów obu ciągów: min(1, 2) = 1. Przepisujemy go i przesuwamy wskaźnik w tym ciągu, z którego wzięliśmy.

> Potem znowu porównujemy pierwsze nieprzepisane elementy: 4 i 2 → bierzemy 2. I tak dalej, aż oba ciągi się skończą.

05 Scalanie krok po kroku

krok	porównujemy	bierzemy	wynik
1	a: 1, b: 2	1 z a	1
2	a: 4, b: 2	2 z b	1 2
3	a: 4, b: 3	3 z b	1 2 3
4	a: 4, b: 4	4 z a (przy remisie — z a)	1 2 3 4
5	a: 9, b: 4	4 z b	1 2 3 4 4
6	a: 9, b: 10	9 z a	1 2 3 4 4 9
7	a skończone	10 z b	1 2 3 4 4 9 **10**

Każdy element przepisujemy raz, więc scalanie ciągów o długościach n i m to tylko **n + m** kroków.

05 Scalanie — kod

REKURENCYJNIE

```
void scal(int i, int j) {
    if (i == n && j == m) return;
    if (j == m || (i < n && a[i] <= b[j])) {
        cout << a[i] << " ";
        scal(i + 1, j);
    } else {
        cout << b[j] << " ";
        scal(i, j + 1);
    }
}
```

PĘTLĄ — W SORTOWANIU PRZEZ SCALANIE

```
int i = l, j = s + 1, k = l;
while (i <= s || j <= r) {
    if (j > r || (i <= s && a[i] <= a[j]))
        pom[k++] = a[i++];
    else
        pom[k++] = a[j++];
}
```

W sortowaniu scalamy dwie **połowy tej samej tablicy** (l…s i s + 1…r) do tablicy pomocniczej pom, a potem kopiujemy wynik z powrotem.

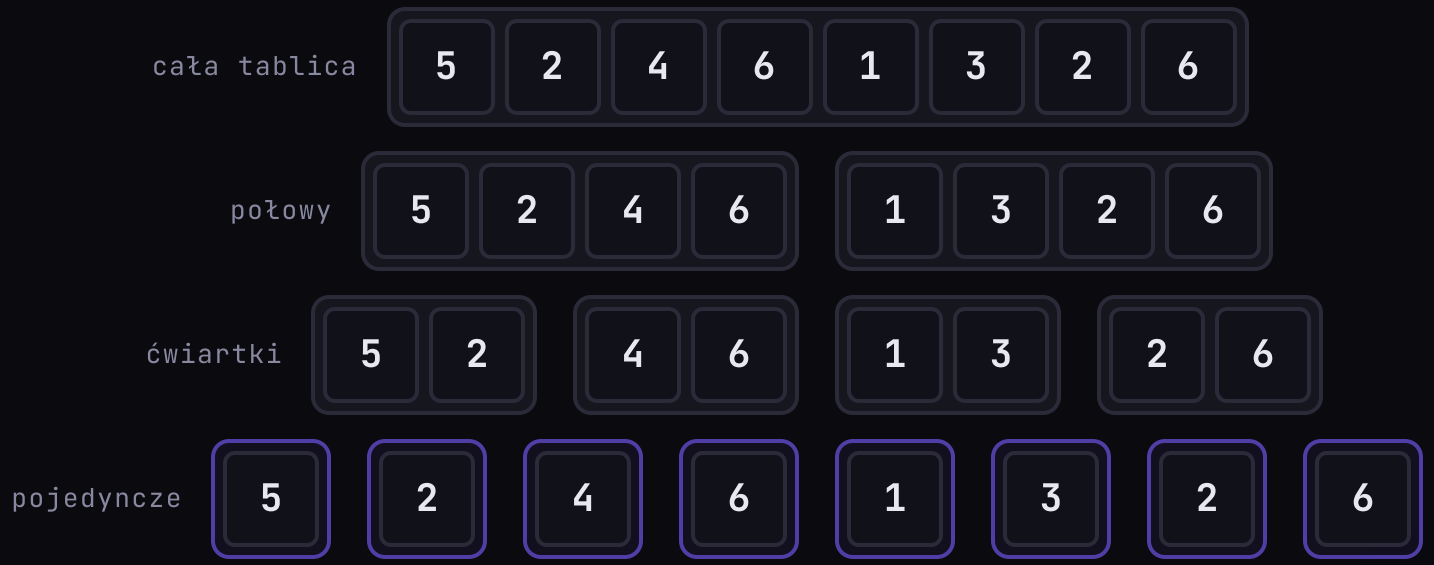
06 Sortowanie przez scalanie (merge sort)

1. **jeden element jest już posortowany — stop,**
2. **podziel tablicę na dwie połowy,**
3. **posortuj każdą połowę rekurencyjnie,**
4. **scal dwie posortowane połowy.**

> Zaufaj rekurencji: zakładamy, że wywołania dla połówek **już umieją sortować**. Nam zostaje tylko scalić ich wyniki — a to umiemy.

Ten algorytm pokazuje Striver w filmie „Merge Sort” z playlisty — zobacz go po tej prezentacji.

06 Dzielenie — w dół



W drodze w dół tylko dzielimy — nic jeszcze nie jest posortowane. Pojedyncze elementy (fioletowe) są posortowane „same z siebie”.

06 Scalanie — w górę



W drodze w górę każde wywołanie **scala** dwie posortowane połówki. Kod scalania jest **po** wywołaniach rekurencyjnych — wykonuje się w drodze powrotnej.

06 Sortowanie przez scalanie — kod

```
int a[100000], pom[100000];

void sortuj(int l, int r) {
    if (l >= r) return;
    int s = (l + r) / 2;
    sortuj(l, s);
    sortuj(s + 1, r);
    int i = l, j = s + 1, k = l;
    while (i <= s || j <= r) {
        if (j > r || (i <= s && a[i] <= a[j]))
            pom[k++] = a[i++];
        else
            pom[k++] = a[j++];
    }
    for (int x = l; x <= r; x++) a[x] = pom[x];
}
```

Wywołanie z main: `sortuj(0, n - 1)`.

1. linijka 4: fragment z 0 lub 1 elementem jest posortowany,
2. linijki 6–7: posortuj połówki (zaufaj rekurencji!),
3. linijki 8–14: scal połówki do pom,
4. linijka 15: przepisz z powrotem do a.

07 Ile pracy wykonuje merge sort?

Spójrz na schemat scalania „w górę”: na **każdym poziomie** scalamy kawałki, które razem mają **n** elementów. To ok. n operacji na poziom.

Poziomów jest $\log_2 n$. Razem: $n \cdot \log_2 n$ operacji.

> Dla n = 100 000 to ok. **1,7 miliona** operacji — ułamek sekundy.

n	n^2 (sortowanie „bąbelkowe”)	$n \cdot \log_2 n$ (merge sort)
1 000	1 000 000	ok. 10 000
100 000	10^{10} — **za wolno**	ok. $1{,}7 \cdot 10^6$
1 000 000	10^{12} — **za wolno**	ok. $2 \cdot 10^7$

07 Dlaczego dzielenie na pół jest tak dobre?

Głębokość log n zamiast n — to różnica między 30 a miliardem.

algorytm	jak maleje problem	złożoność
suma tablicy (część 2)	n → n - 1, jedno wywołanie	O(n)
Fibonacci bez pamięci (część 3)	n → n - 1 i n - 2	$O(2^n)$
wyszukiwanie binarne	n → n / 2, jedno wywołanie	**O(log n)**
szybkie potęgowanie	n → n / 2, jedno wywołanie	**O(log n)**
merge sort	n → dwa razy n / 2 + scalanie O(n)	**O(n log n)**

08 Inwersje

Inwersja to para elementów stojących „w złej kolejności”: większy przed mniejszym, czyli $i < j$ i $a_i > a_j$.

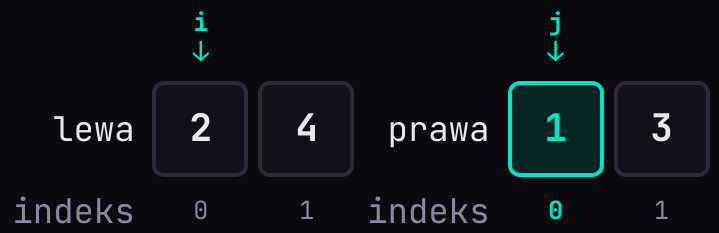


Inwersje: (2, 1), (4, 1), (4, 3) — razem **3**.

Liczba inwersji mówi, jak bardzo tablica jest „nieposortowana”: posortowana ma 0 inwersji, odwrócona — najwięcej, $n(n - 1) / 2$.

> Sprawdzanie każdej pary to $n^2 / 2$ porównań — dla $n = 10^5$ za dużo. Da się szybciej: **podczas sortowania przez scalanie**. (Film Strivera „Count Inversions”.)

08 Inwersje policzymy przy scalaniu



Bierzemy **1** z prawej połowy. Jest mniejsze od **wszystkich** elementów, które zostały w lewej: 2 i 4. To **2 inwersje** naraz!

Każda inwersja jest albo w lewej połowie, albo w prawej (te policzą wywołania rekurencyjne), albo **„pomiędzy”**: większy element w lewej, mniejszy w prawej.

> Te „pomiędzy” liczymy przy scalaniu: gdy bierzemy element z prawej, dodajemy liczbę elementów **pozostałych w lewej** połowie: `s - i + 1`.

08 Zliczanie inwersji — kod

```
long long sortuj(int l, int r) {
    if (l >= r) return 0;
    int s = (l + r) / 2;
    long long w = sortuj(l, s) + sortuj(s + 1, r);
    int i = l, j = s + 1, k = l;
    while (i <= s || j <= r) {
        if (j > r || (i <= s && a[i] <= a[j]))
            pom[k++] = a[i++];
        else {
            w += s - i + 1;
            pom[k++] = a[j++];
        }
    }
    for (int x = l; x <= r; x++) a[x] = pom[x];
    return w;
}
```

To **merge sort** z jedną dodatkową linijką (10) i wynikiem zwracanym przez return.

1. inwersje w lewej połowie + inwersje w prawej (linijka 4),
2. plus inwersje „pomiędzy”, liczone przy scalaniu (linijka 10).

Uwaga: Przy remisie (a[i] == a[j]) bierz z **lewej** — równe liczby nie tworzą inwersji. Wynik może przekroczyć int.

09 Quicksort — inny sposób dzielenia

Merge sort dzieli „na ślepo” na pół, a cała praca jest przy **łączeniu**. Quicksort robi odwrotnie: cała praca jest przy **dzieleniu**.

1. wybierz element osiowy — **pivot**,
2. rozdziel elementy na **mniejsze**, **równe** i **większe** od pivota,
3. posortuj rekurencyjnie mniejsze i większe,
4. połącz: mniejsze, równe, większe — i gotowe.



Quicksort pokazuje Striver w filmie „Quick Sort For Beginners”. Zwykle jest bardzo szybki, ale przy pechowym pivocie (np. zawsze najmniejszy element) zwalnia do n^2 — dlatego pivot losuje się albo bierze ze środka.

09 Quickselect: k-ty najmniejszy element

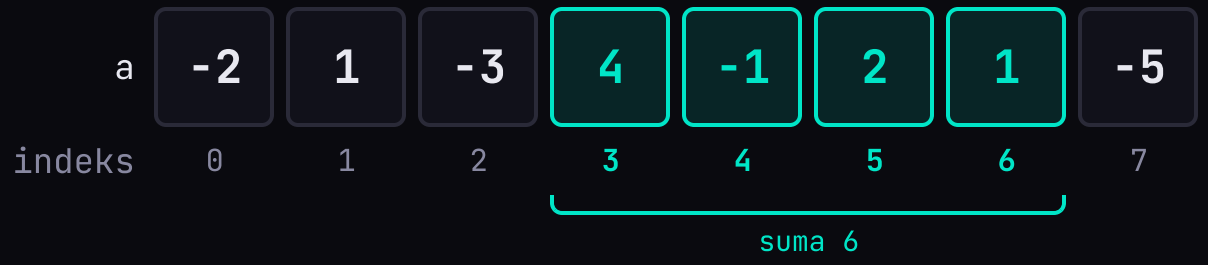
```
int kty(vector<int> v, int k) {
    int pivot = v[v.size() / 2];
    vector<int> mniejsze, wieksze;
    int rowne = 0;
    for (int x : v) {
        if (x < pivot) mniejsze.push_back(x);
        else if (x > pivot) wieksze.push_back(x);
        else rowne++;
    }
    if (k <= mniejsze.size()) return kty(mniejsze, k);
    if (k <= mniejsze.size() + rowne) return pivot;
    return kty(wieksze, k - mniejsze.size() - rowne);
}
```

Nie musimy sortować wszystkiego, żeby znaleźć k-ty najmniejszy element. Po podziale wiemy, **w której części** on jest — i tylko tam szukamy dalej.

k	gdzie jest k-ty?
k ≤ |mniejsze|	w mniejszych
dalej, do |m| + |r|	to sam pivot
jeszcze dalej	w większych (k przesuwamy)

10 Najlepszy spójny fragment

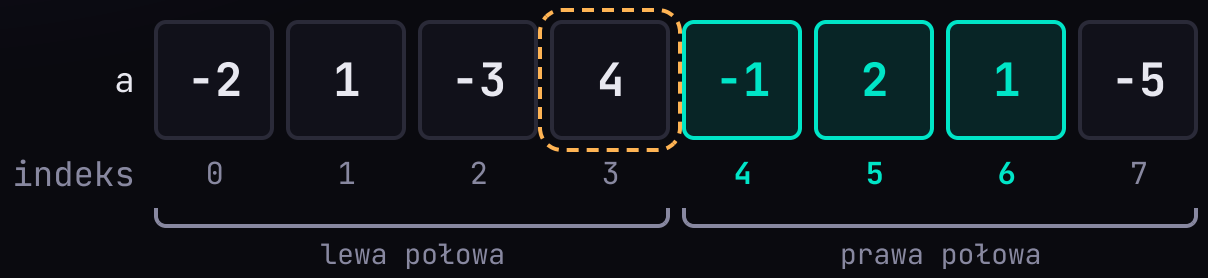
Wybierz **spójny** fragment tablicy (kilka liczb obok siebie) o **największej sumie**.



Podziel tablicę na pół. Najlepszy fragment leży:

1. cały w **lewej** połowie — znajdzie go wywołanie dla lewej,
2. cały w **prawej** — wywołanie dla prawej,
3. **przez środek** — kawałek kończący się na końcu lewej połowy + kawałek zaczynający się na początku prawej.

10 Fragment przez środek



kierunek	sumy kolejnych kawałków	najlepsza
od środka w lewo	a[3] = 4, 4 − 3 = 1, 1 + 1 = 2, 2 − 2 = 0	**4**
od środka w prawo	a[4] = −1, −1 + 2 = 1, 1 + 1 = 2, 2 − 5 = −3	**2**

Najlepszy fragment przez środek = najlepsza suma w lewo + najlepsza suma w prawo = 4 + 2 = **6**.

Porównujemy go z najlepszymi fragmentami wewnątrz połówek (te znajdzie rekurencja) i bierzemy największy z trzech.

10 Najlepszy fragment — kod

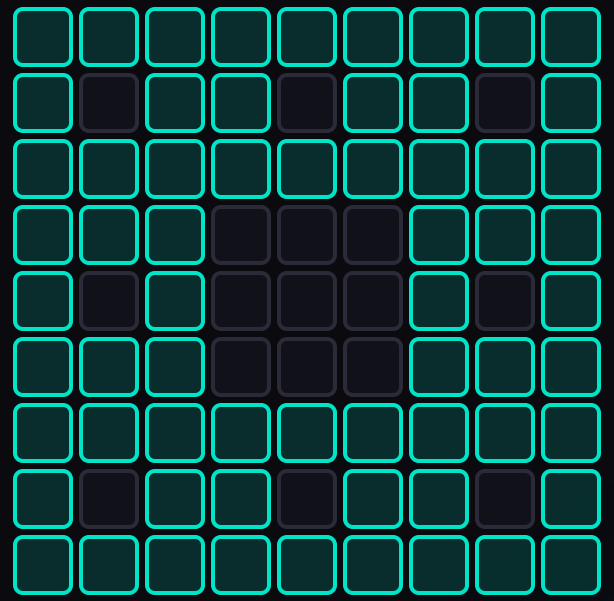
```
long long najlepszy(int l, int r) {
    if (l == r) return a[l];
    int s = (l + r) / 2;
    long long w = max(najlepszy(l, s), najlepszy(s + 1, r));
    long long suma = 0, lewa = LLONG_MIN, prawa = LLONG_MIN;
    for (int i = s; i >= l; i--) { suma += a[i]; lewa = max(lewa, suma); }
    suma = 0;
    for (int i = s + 1; i <= r; i++) { suma += a[i]; prawa = max(prawa, suma); }
    return max(w, lewa + prawa);
}
```

Na każdym poziomie rekurencji pętle przechodzą łącznie przez całą tablicę, a poziomów jest log n — razem **O(n log n)**. (Istnieje też sprytniejszy sposób O(n) — algorytm Kadane; w kompendium: „Maksymalna suma na przedziale 1D”.)

11 Fraktal: dywan Sierpińskiego

Dywan rzędu 0 to jedno pole. Dywan rzędu k to **3 × 3 dywany rzędu k − 1**, ze środkowym wyciętym. Rekurencja rysuje się tu sama!

```
void rysuj(int k, int w, int x) {
    if (k == 0) { t[w][x] = '#'; return; }
    int b = pot3[k - 1];          // bok mniejszego dywanu
    for (int i = 0; i < 3; i++)
        for (int j = 0; j < 3; j++)
            if (!(i == 1 && j == 1))
                rysuj(k - 1, w + i * b, x + j * b);
}
```



11 Napis Fibonacciego: k-ta litera bez budowania napisu

S(1) = **a**, S(2) = **b**, S(n) = S(n − 1) + S(n − 2): b, ba, bab, babba, babbabab, … S(80) ma ponad 10^{16} liter — nie da się go zbudować.

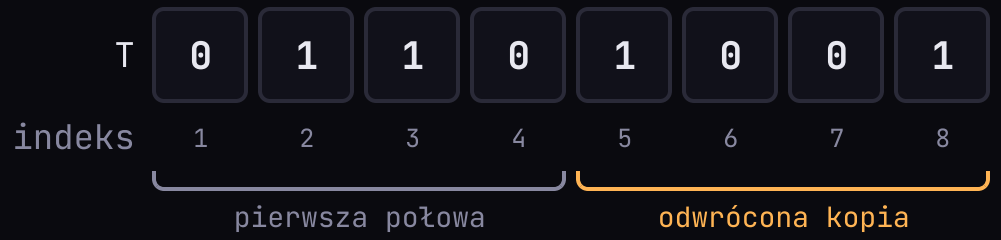
Ale k-ta litera S(n) leży **albo w części S(n − 1)** (gdy k ≤ długość S(n − 1)), **albo w części S(n − 2)** — na pozycji k − długość S(n − 1). Schodzimy tylko w jedną część!

```
char litera(int n, long long k) {
    if (n == 1) return 'a';
    if (n == 2) return 'b';
    if (k <= dl[n - 1]) return litera(n - 1, k);
    return litera(n - 2, k - dl[n - 1]);
}
```

dl[n] — długości napisów, policzone wcześniej pętlą: dl[n] = dl[n − 1] + dl[n − 2]. To znowu liczby Fibonacciego!

11 Ciąg Thuego–Morse'a

0 → 01 → 0110 → 01101001 → … Każdy krok dokleja kopię z zamienionymi cyframi.



k-ta cyfra w drugiej połowie (długości 2p) to **odwrócona** cyfra z pozycji k − p.

```
int cyfra(long long k) {
    if (k == 1) return 0;
    long long p = 1;
    while (2 * p < k) p *= 2;
    return 1 - cyfra(k - p);
}
```

Na przykład 6. cyfra: p = 4, więc to odwrócona 2. cyfra; 2. cyfra to odwrócona 1., czyli 1 − 0 = 1; zatem 6. cyfra = 1 − 1 = **0**.

12 Pytanie 1: ile wywołań zrobi `potega(2, 1000)`?

```
long long potega(long long a, long long n) {
    if (n == 0) return 1 % m;
    long long p = potega(a, n / 2);
    p = p * p % m;
    if (n % 2 == 1) p = p * (a % m) % m;
    return p;
}
```

NA EKRANIE

?

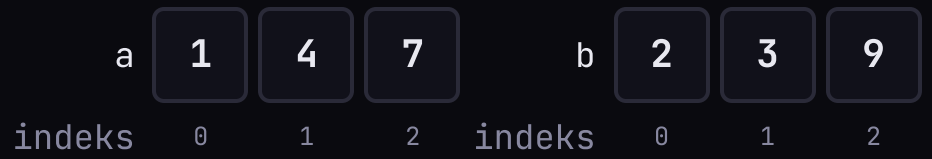
Zastanów się i zapisz odpowiedź, **zanim** przejdziesz dalej.

12 Odpowiedź 1: 11

n idzie: 1000 → 500 → 250 → 125 → 62 → 31 → 15 → 7 → 3 → 1 → 0. To **11** wywołań ($\log_2 1000 \approx 10$, plus warunek stopu).

Zwykłe potęgowanie z części 2 potrzebowałoby 1001 wywołań.

12 Pytanie 2: co wyjdzie ze scalenia?



NA EKRANIE

?

Zastanów się i zapisz odpowiedź, **zanim** przejdziesz dalej.

12 Odpowiedź 2: 1 2 3 4 7 9

NA EKRANIE

1 2 3 4 7 9

1 (a) · 2 (b) · 3 (b) · 4 (a) · 7 (a) · 9 (b). Sześć kroków — każdy element przepisany raz.

12 Pytanie 3: ile inwersji ma tablica 3 1 2?



NA EKRANIE

?

Zastanów się i zapisz odpowiedź, **zanim** przejdziesz dalej.

12 Odpowiedź 3: 2



(3, 1) i (3, 2). Para (1, 2) jest w dobrej kolejności. Merge sort dzieli na [3, 1] i [2]: scalając [3] z [1] bierze 1, gdy w lewej czeka 3 → +1; scalając [1, 3] z [2] bierze 2, gdy w lewej czeka 3 → +1. Razem 2.

12 Pytanie 4: dlaczego to jest wolne?

```
long long potega(long long a, long long n) {
    if (n == 0) return 1 % m;
    long long p = potega(a, n / 2) * potega(a, n / 2) % m;
    if (n % 2 == 1) p = p * (a % m) % m;
    return p;
}
```

NA EKRANIE

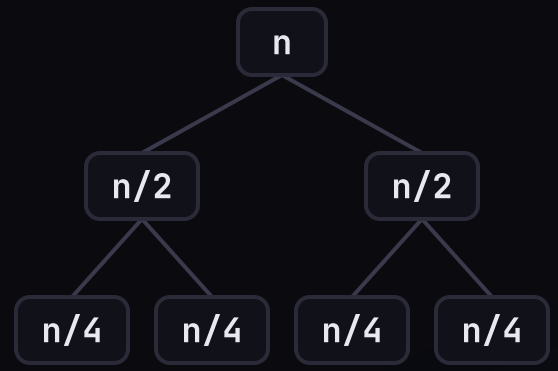
?

Zastanów się i zapisz odpowiedź, **zanim** przejdziesz dalej.

12 Odpowiedź 4: dwa wywołania zamiast jednego

Uwaga: Każde wywołanie woła **dwa** mniejsze, a każde z nich znowu dwa… Drzewo ma log n poziomów, ale na każdym **dwa razy więcej** węzłów: razem ok. **2n** wywołań. Zysk z dzielenia na pół zniknął!

Policz `potega(a, n / 2)` raz, zapisz w zmiennej i pomnóż przez siebie. Ta sama lekcja co przy Fibonaccim: **nie licz dwa razy tego samego**.



13 Ściąga: dziel i zwyciężaj

Algorytm	Podział	Łączenie	Złożoność
min / max	dwie połowy	min z min, max z max	O(n)
szybkie potęgowanie	$a^n \to a^{n/2}$	p · p (· a)	O(log n)
wyszukiwanie binarne	jedna połowa	— (odrzucamy drugą)	O(log n)
merge sort	dwie połowy	scalanie dwóch ciągów	O(n log n)
inwersje	jak merge sort	scalanie + s − i + 1	O(n log n)
quickselect	podział wg pivota	— (jedna część)	zwykle O(n)
najlepszy fragment	dwie połowy	max z połówek i przez środek	O(n log n)

13 Zadania na koniec — Twoja kolej

Zadania są na **t-olimpians.com** → **Kompendium** → **Funkcje rekurencyjne** → **Krok 6**. Każde ma sprawdzarkę i wskazówkę:

1. Minimum i maksimum przez połowienie
2. Szybkie potęgowanie
3. Wyszukiwanie binarne
4. Scalanie dwóch ciągów
5. Sortowanie przez scalanie
6. Zliczanie inwersji
7. K-ty najmniejszy element
8. Najlepszy spójny fragment
9. Dywan Sierpińskiego
10. Napis Fibonacciego
11. Ciąg Thuego–Morse'a

To koniec kursu rekurencji. Dalej w kompendium: **Memoizacja i tabulacja** (programowanie dynamiczne), **Dziel i zwyciężaj** oraz **Wyszukiwanie binarne po wyniku**.

// KONIEC KURSU REKURENCJI

Rekurencja nie ma przed Tobą tajemnic!

Od odliczania przez memoizację, generowanie i backtracking aż po dziel i zwyciężaj — sześć kroków za Tobą. Zrób zadania z kroku 6 i ruszaj dalej: programowanie dynamiczne czeka.

